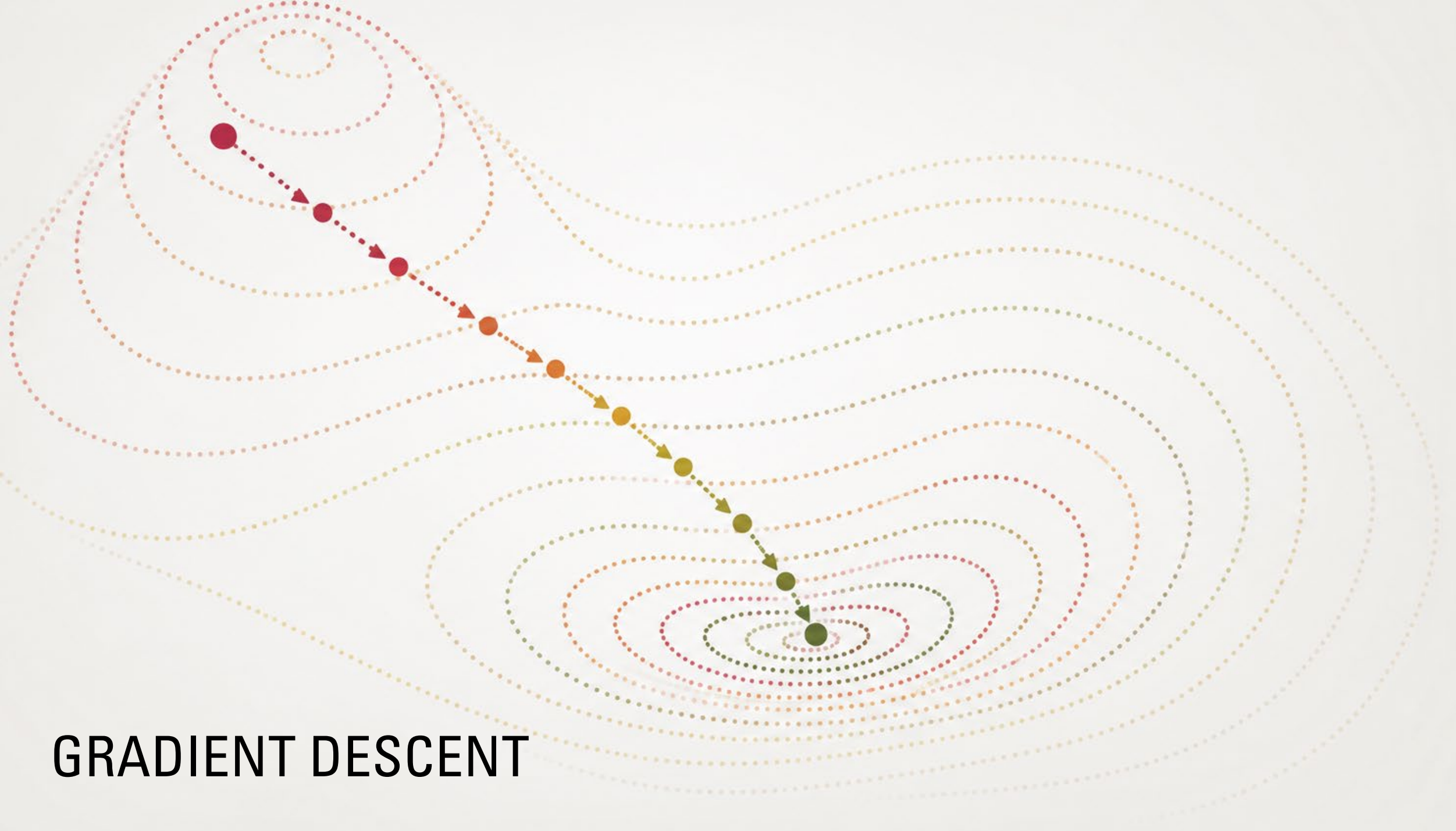




**GRADIENT DESCENT**

---

# OPTIMIZATION IN MACHINE LEARNING

- Many (all?) machine learning problems can be posed as a minimization or maximization problem
- In some cases, a closed-form solution exists
  - E.g., linear regression
- Most of the time, a closed-form (analytic) solution doesn't exist
  - E.g., logistic regression
- We can solve these problems using optimization theory

---

# UNCONSTRAINED OPTIMIZATION

- An *unconstrained optimization problem* has the form

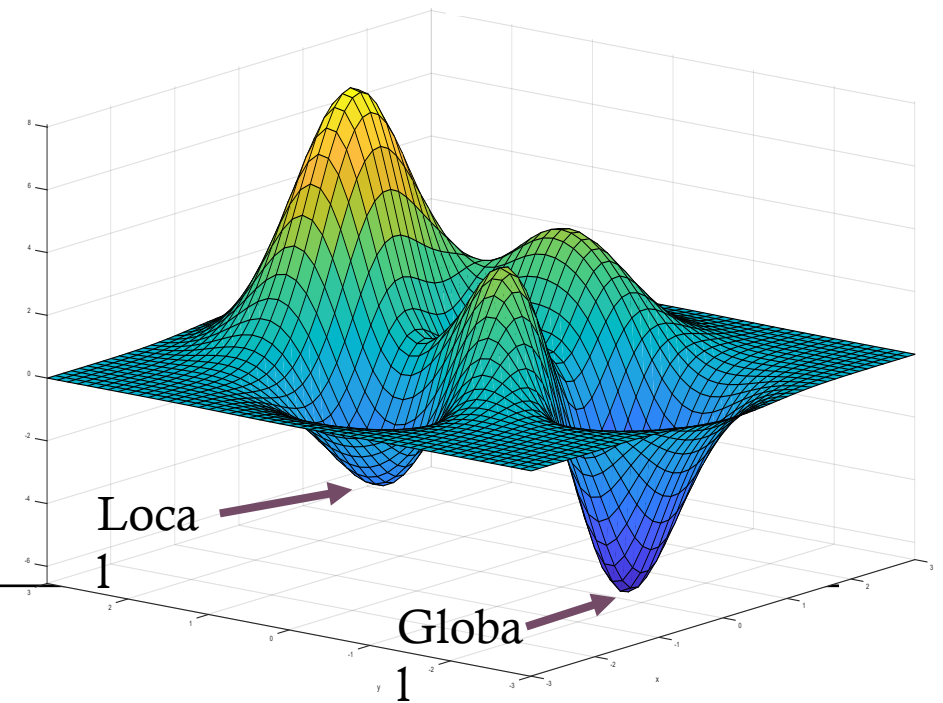
$$\min_{\mathbf{x} \in \mathbb{R}^d} f(\mathbf{x})$$

where  $f: \mathbb{R}^d \rightarrow \mathbb{R}$  is called the *objective function*.

- What about maximization?
- A point  $\mathbf{x}^* \in \mathbb{R}^d$  is called a *local minimizer* if  $\exists r > 0$  such that
  - $f(\mathbf{x}^*) \leq f(\mathbf{x}) \forall \mathbf{x}$  satisfying  $\|\mathbf{x} - \mathbf{x}^*\| < r$
- $\mathbf{x}^*$  is called a *global minimizer* if
  - $f(\mathbf{x}^*) \leq f(\mathbf{x}) \forall \mathbf{x} \in \mathbb{R}^d$
- Is a global minimizer also a local minimizer?

$\exists$  = “there exists”

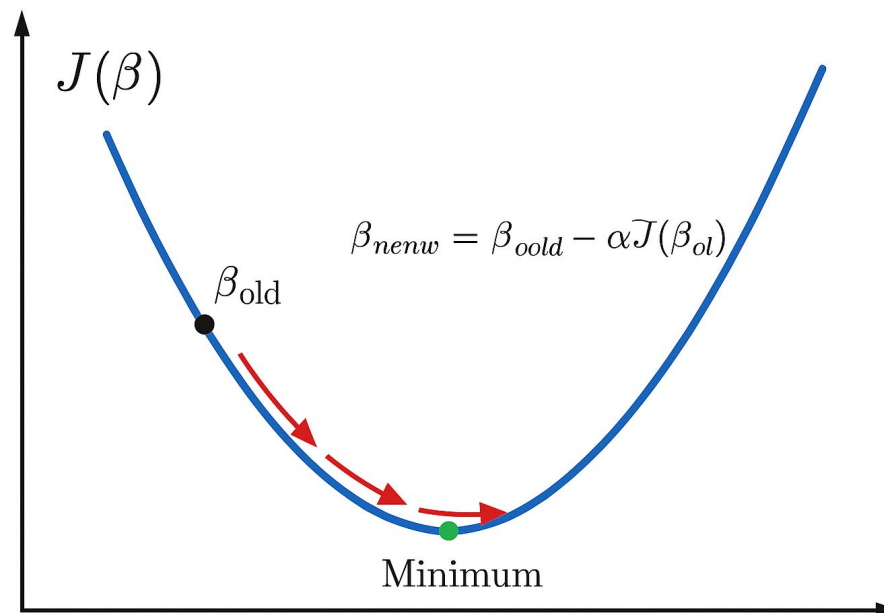
$\forall$  = “for all”



---

# GRADIENT DESCENT (HIGH LEVEL)

- Numerical optimization algorithm that tweaks parameters iteratively until the minimum is found
  - “Descend” the gradient of the cost function in the steepest direction until the minimum is reached
- What is the gradient?



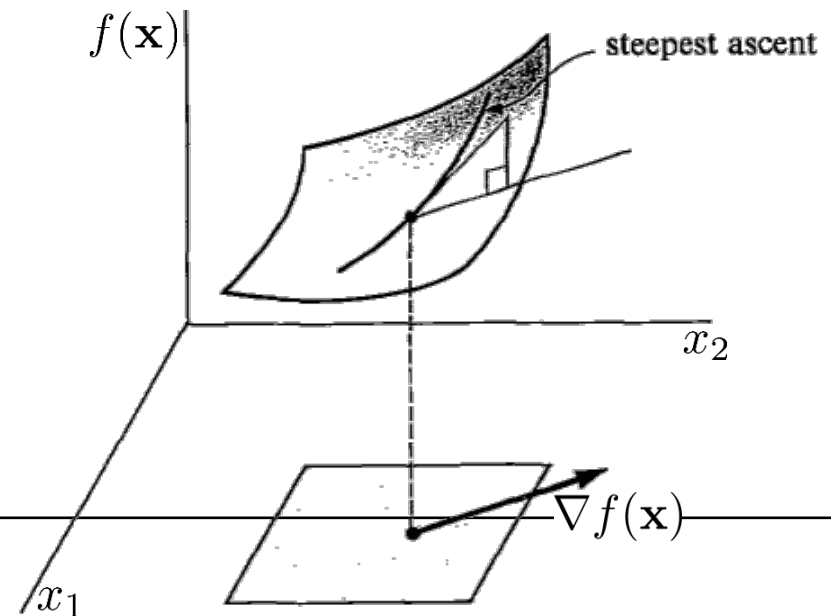
---

# GRADIENT

- Given a function  $f: \mathbb{R}^d \rightarrow \mathbb{R}$ , the *gradient* of  $f$  at  $\mathbf{x} = [x_1 \dots x_d]^T \in \mathbb{R}^d$  is defined by

$$\nabla f(\mathbf{x}) := \begin{bmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \vdots \\ \frac{\partial f(\mathbf{x})}{\partial x_d} \end{bmatrix}$$

- If the gradient exists for all  $\mathbf{x} \in \mathbb{R}^d$ , we say  $f$  is *differentiable*
- The gradient gives the direction of *steepest ascent*



---

# FIRST ORDER NECESSARY CONDITION

- If  $f$  is differentiable and  $\mathbf{x}^*$  is a local minimizer of  $f$ , then
$$\nabla f(\mathbf{x}^*) = \mathbf{0}.$$
  - Note that this condition is necessary but not sufficient for  $\mathbf{x}^*$  to be a local minimizer
    - Why?
  - If  $\nabla f(\mathbf{x}) = 0$  for some  $\mathbf{x}$ , then  $\mathbf{x}$  is said to be a critical point or a stationary point of  $f$
-

---

# HESSIAN

- The Hessian of  $f$  at  $\mathbf{x}$  is the  $d \times d$  matrix

$$\nabla^2 f(\mathbf{x}) := \begin{bmatrix} \frac{\partial^2 f(\mathbf{x})}{\partial x_1^2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_1 \partial x_d} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f(\mathbf{x})}{\partial x_d \partial x_1} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial x_d^2} \end{bmatrix}$$

- We say that  $f$  is twice differentiable if  $\nabla^2 f(\mathbf{x})$  exists  $\forall \mathbf{x} \in \mathbb{R}^d$
- We say  $f$  is twice continuously differentiable if it is twice differentiable and all of the second derivatives are continuous functions of  $\mathbf{x}$
- If  $f$  is twice continuously differentiable, then  $\nabla^2 f(\mathbf{x})$  is a symmetric matrix  $\forall \mathbf{x}$ , i.e.,

$$\frac{\partial^2 f(\mathbf{x})}{\partial x_i \partial x_j} = \frac{\partial^2 f(\mathbf{x})}{\partial x_j \partial x_i}$$

$$\forall \mathbf{x} \in \mathbb{R}^d, \forall i, j = 1, \dots, d$$

---



---

# POSITIVE (SEMI-)DEFINITE MATRICES

- Let  $A$  be a  $d \times d$  matrix. We say that  $A$  is *positive definite* (PD) if  $\mathbf{x}^T A \mathbf{x} > 0$  for all  $\mathbf{x} \neq \mathbf{0}$ . We say that  $A$  is *positive semi-definite* (PSD) if  $\mathbf{x}^T A \mathbf{x} \geq 0$  for all  $\mathbf{x} \in \mathbb{R}^d$ .
  - PD and PSD arise frequently in ML, for example
    - Gram matrices (i.e.,  $XX^T$ )
    - Kernel matrices
    - Covariance matrices
    - Hessian matrices (sometimes)
  - PD/PSD matrices are not necessarily symmetric, e.g.
$$\begin{bmatrix} x \\ y \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x & y \end{bmatrix} = x^2 + y^2$$
    - However, we will only consider PD/PSD matrices that are also symmetric
-



---

# SECOND ORDER NECESSARY CONDITION

- If  $f$  is twice continuously differentiable and  $\mathbf{x}^*$  is a local min, then  $\nabla^2 f(\mathbf{x}^*)$  is positive semi-definite, i.e.,

$$\mathbf{z}^T \nabla^2 f(\mathbf{x}^*) \mathbf{z} \geq 0, \quad \forall \mathbf{z} \in \mathbb{R}^d$$

- This generalizes the result from single-variable calculus that the second derivative is nonnegative at a local min
- Note: the converse is *not* true! (If PSD, then local min.. not guaranteed!)
  - However, if not PSD, then not min!



---

# LEADING PRINCIPAL MINORS

- Assume  $A$  is a real symmetric  $n \times n$  matrix.
  - The  $k$ th leading principal minor is  $\Delta_k = \det(A_{1:k, 1:k})$ , the determinant of the upper-left  $k \times k$  block.
  - **PD (Sylvester's criterion):**  $A$  is positive definite if and only if  $\Delta_1 > 0, \Delta_2 > 0, \dots, \Delta_n > 0$ .
  - **PSD:**  $A$  is positive semidefinite if and only if every principal minor is nonnegative.
    - A principal minor uses the same subset of row and column indices, not necessarily the first  $k$ .
  - **Caution:**  $\Delta_k \geq 0$  (leading) for every  $k$  is insufficient for PSD. For example,  $\text{diag}(0, -1)$  has leading minors 0 and 0, but is not PSD.
-



---

# WORKED EXAMPLE: LEADING MINORS

- Consider the real symmetric  $2 \times 2$  matrix

$$A(t) = \begin{bmatrix} 2 & 1 \\ 1 & t \end{bmatrix} \quad (t \text{ is real})$$

- **First leading minor:**  $\Delta_1 = \det([2]) = 2 > 0$ .
- **Second leading minor:** use  $ad - bc$  for the determinant.

$$\Delta_2 = \det(A(t)) = \begin{vmatrix} 2 & 1 \\ 1 & t \end{vmatrix} = (2)(t) - (1)(1) = 2t - 1$$

- **PD conclusion:** both leading minors must be positive.  
Since  $\Delta_1 = 2 > 0$ , we need  $2t - 1 > 0$ , giving  $t > 1/2$ .
  - **For example,  $t = 1$ :**  $\det(A(1)) = 2(1) - 1 = 1 > 0$ ,  
so  $A(1)$  is PD.
-



---

# WORKED EXAMPLE: THE PSD CHECK

- For  $A(t)$ , the three principal minors are  $a_{11} = 2$ ,  $a_{22} = t$ , and  $\det(A(t)) = 2t - 1$ . PSD requires all three to be  $\geq 0$ .
- At the boundary  $t = 1/2$ :

$$A(1/2) = \begin{bmatrix} 2 & 1 \\ 1 & 1/2 \end{bmatrix}$$

$$\det(A(1/2)) = \begin{vmatrix} 2 & 1 \\ 1 & 1/2 \end{vmatrix} = (2)(1/2) - (1)(1) = 1 - 1 = 0$$

- **PSD check:** the principal minors are 2,  $1/2$ , and 0. All are nonnegative, so  $A(1/2)$  is PSD. The zero determinant means it is not PD.
  - **For  $t < 1/2$ :**  $\det(A(t)) = 2t - 1 < 0$ , so  $A(t)$  is neither PD nor PSD.
  - **Summary:**  $A(t)$  is PD for  $t > 1/2$  and PSD for  $t \geq 1/2$ .
-

---

# GROUP EXERCISE

Notation:

$$\boldsymbol{x} = \begin{bmatrix} x \\ y \end{bmatrix}$$

Consider the function

$$f(x, y) = x^2 + 4xy - y^2 - 8x - 6y + 10$$

1. Determine  $\nabla f(x, y)$
  2. Determine  $\nabla^2 f(x, y)$
  3. Determine a critical point  $\boldsymbol{x}^*$
  4. Is  $\boldsymbol{x}^*$  a local min, a local max, or neither?
-

---

# CONVEXITY

- We say that  $f$  is *convex* if

$$f(t\mathbf{x} + (1 - t)\mathbf{y}) \leq tf(\mathbf{x}) + (1 - t)f(\mathbf{y})$$

for all  $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$  and  $t \in [0,1]$ .

- We say  $f$  is *strictly convex* if

$$f(t\mathbf{x} + (1 - t)\mathbf{y}) < tf(\mathbf{x}) + (1 - t)f(\mathbf{y})$$

$\forall \mathbf{x} \neq \mathbf{y}$  and  $t \in (0,1)$ .

---

---

# CONVEX FUNCTIONS ARE NICE

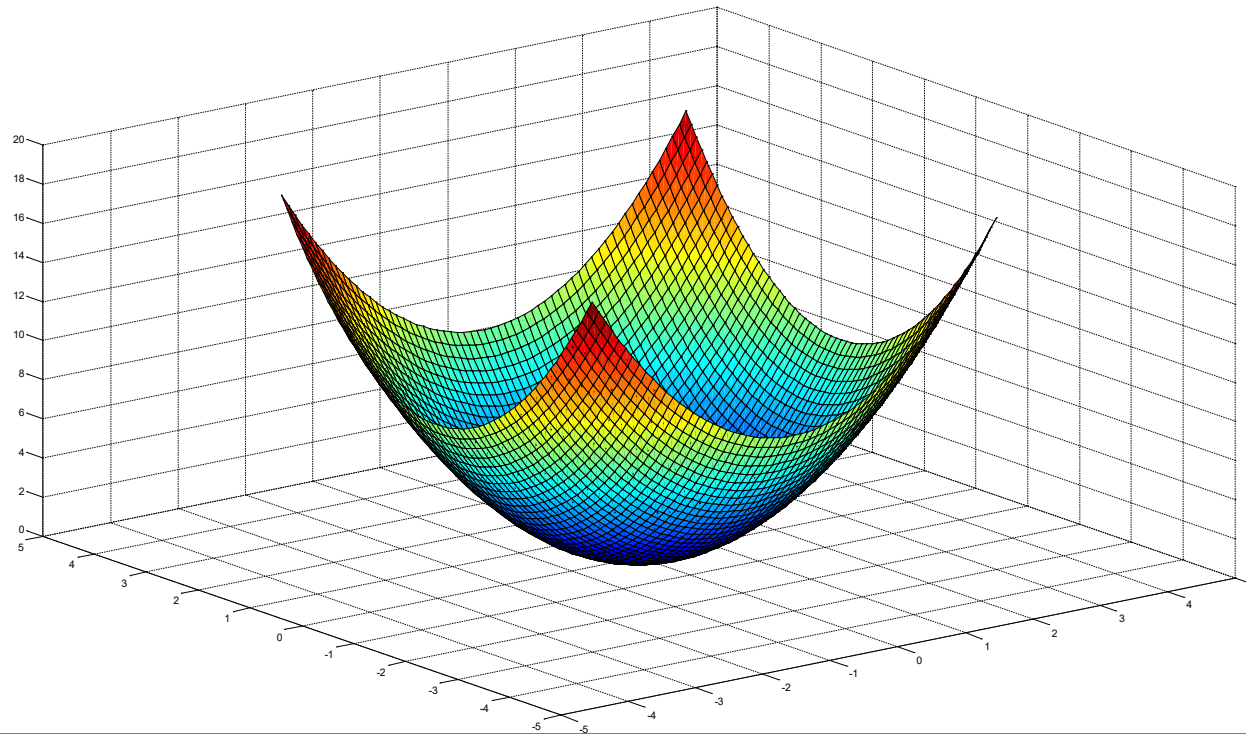
Properties of convex functions

1. If  $f$  is convex, then every local min is a global min
2. If  $f$  is strictly convex, then  $f$  has at most one global min

---

# CONVEXITY EXAMPLE

$$f(x, y) = x^2 + y^2$$





---

# GROUP EXERCISE

1. Give an example of a function  $f$  that is
    - convex but not strictly convex
    - convex and has more than one global minimum
    - strictly convex, but has no global minimum
  2. Is the sum of convex functions necessarily convex? Prove or provide a counter example
  3. Is the product of convex functions necessarily convex? Prove or provide a counter example
-

---

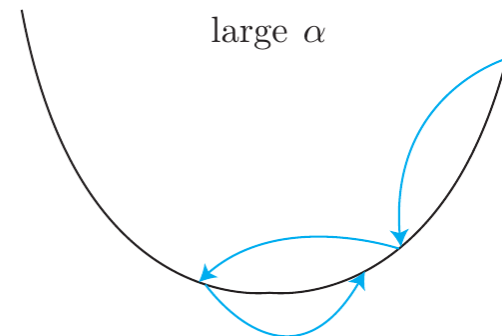
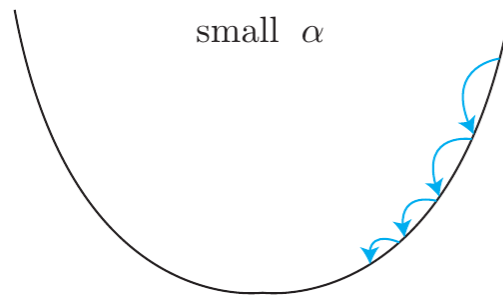
# GRADIENT DESCENT (GD)

- Consider minimizing the generic objective function  $J(\boldsymbol{\theta})$
- What is a geometric interpretation of  $\nabla J(\boldsymbol{\theta})$ ?
  - Direction of steepest ascent
  - Thus iterative approaches take steps in opposite direction, i.e., the direction of steepest descent
- Initial guess  $\boldsymbol{\theta}_0$
- Learning rate,  $\alpha$
- For  $t = 1, \dots, \text{max\_iter}$

$$\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1} - \alpha \nabla J(\boldsymbol{\theta}_{t-1})$$

If convergence condition satisfied, exit

End



---

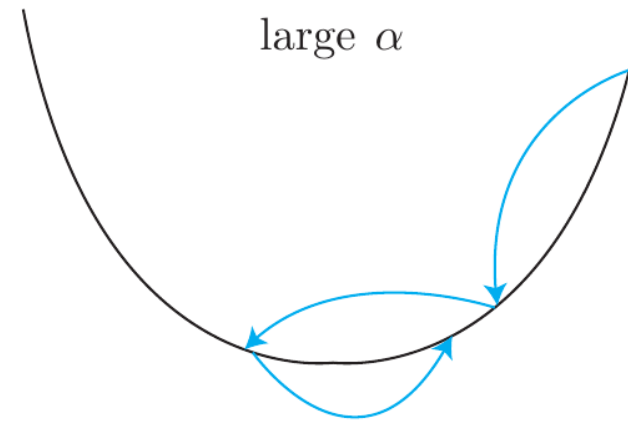
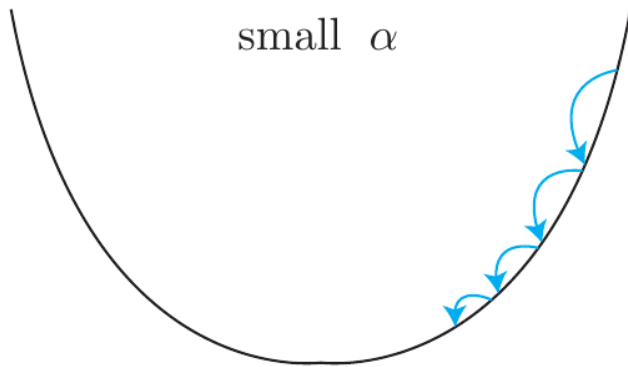
# LEARNING RATE

- If the learning rate is too high, the gradient descent algorithm will bounce around the minimum and never converge
  - (See Figure 4-5 in HOML)
- If the learning rate is too low, it could take a very long time to find the minimum
  - (See Figure 4-4 in HOML)
- It is common to start with a small learning rate ( $\sim 0.01$ ) and adjust based on the performance of the model

---

# LEARNING RATE AS A FUNCTION

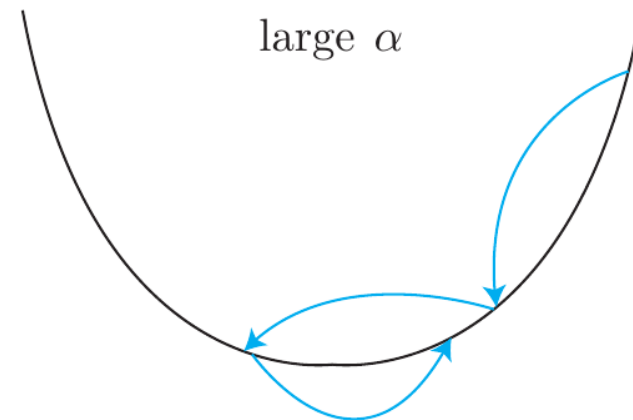
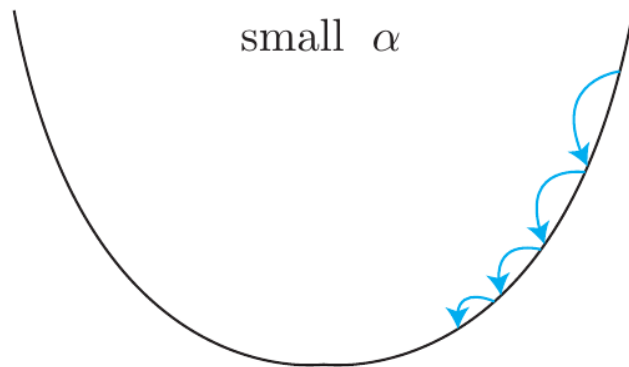
- In practice, typically need  $\alpha$  to decrease as a function of  $t$ 
  - Example:  $\alpha_t = \frac{1}{t}$



---

# GROUP EXERCISE

1. What would be a good convergence condition (also known as a stopping criterion) for terminating GD?
2. What are some potential problems with setting  $\alpha_t = \frac{1}{t}$ ?
3. How could you choose  $\alpha_t$  to counter these problems?
4. Does the initialization  $\theta_0$  affect the result?





---

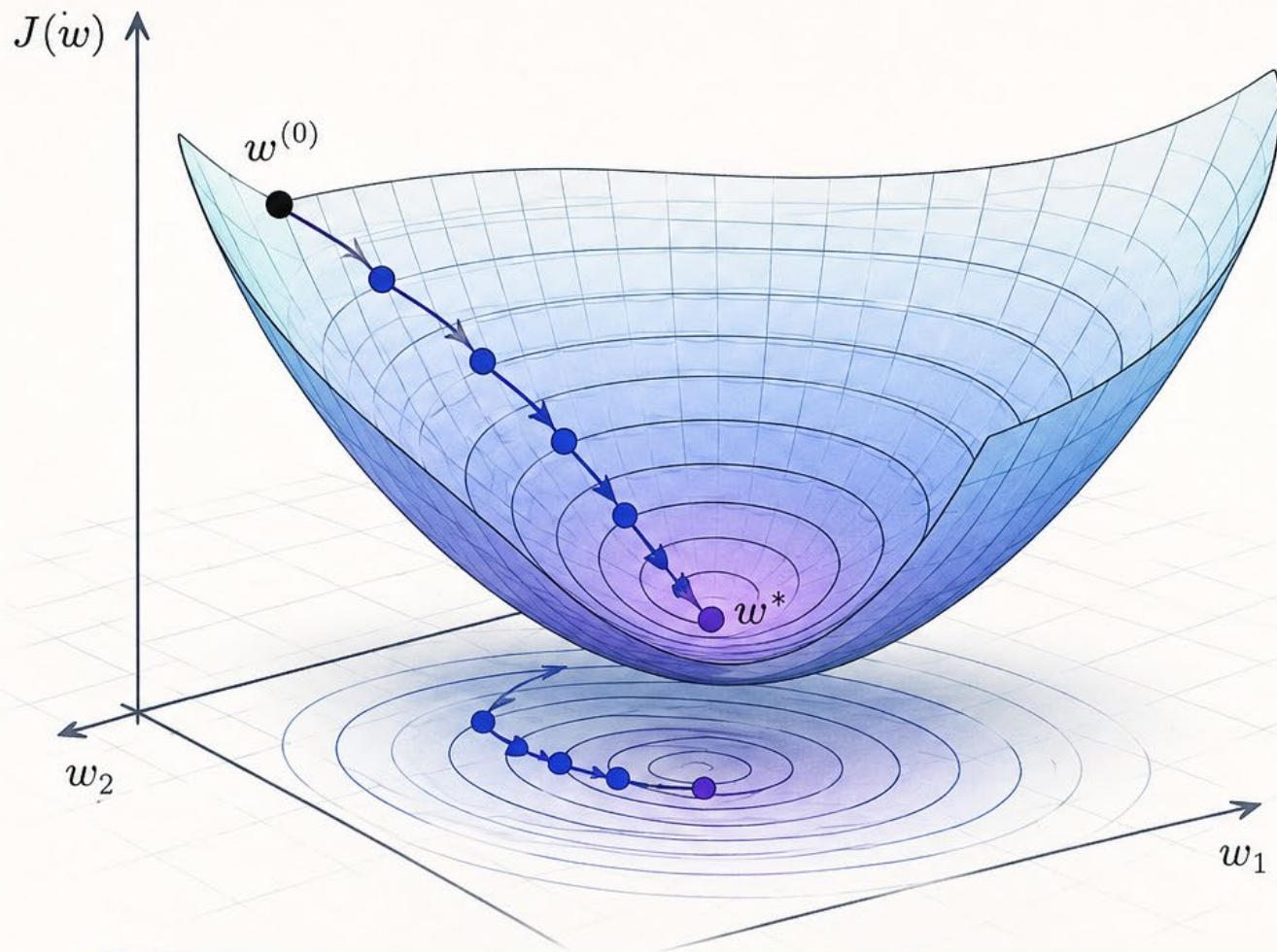
# GROUP EXERCISE SOLUTIONS

1. What would be a good convergence condition (also known as a stopping criterion) for terminating GD?
    - If the norm of the gradient is small
    - If the decrease in the objective function or the parameter values is small
    - A fixed number of iterations
  2. What are some potential problems with setting  $\alpha_t = \frac{1}{t}$ ?
    - The step size may decay too rapidly
  3. How could you choose  $\alpha_t$  to counter these problems?
    - Could choose  $\alpha_t$  adaptively; e.g., use momentum
  4. Does the initialization  $\theta_0$  affect the result?
    - If  $J$  is convex, we will eventually find the global min although, the initialization may affect the number of iterations required. If  $J$  is not convex, we may end up at a local min instead
-

---

# LEARNING SCHEDULE

- A way to help “settle” stochastic gradient descent
  - Gradually reduce the learning rate according to a function called the “learning schedule”
  - Additional Points:
    - Common schedules include:
      - Step decay: Reduce learning rate by a factor after a fixed number of epochs.
      - Exponential decay: Multiply learning rate by a constant factor each epoch.
      - Polynomial decay: Decrease learning rate following a polynomial function.
  - Modern techniques:
    - Cosine Annealing: Learning rate follows a cosine curve, often combined with restarts.
    - One-cycle policy: Increase learning rate initially, then decrease sharply for better convergence.
    - Adaptive methods (Adam, RMSProp): Adjust learning rate per parameter automatically based on gradient history.
-



Update rule:

$$w^{(t+1)} = w^{(t)} - \eta \nabla J(w^{(t)})$$

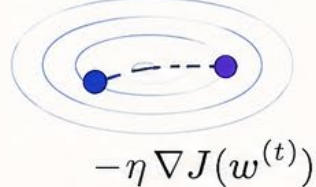
- $w^{(0)}$  Initial parameters
- $w^{(t)}$  Current parameters
- $w^*$  Optimal parameters (minimum)
- Path of gradient descent

$$\nabla J(w) = \begin{bmatrix} \frac{\partial J}{\partial w_1} \\ \frac{\partial J}{\partial w_2} \end{bmatrix} \quad \text{Gradient (direction of steepest ascent)}$$

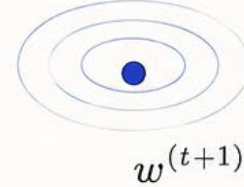
At each step  $t$ : 1. Compute gradient



2. Take a step



3. Arrive at new point



Repeat until convergence  
(when updates become very small)





---

# STOCHASTIC GRADIENT DESCENT (SGD)

- In many ML problems, we can write

$$J(\boldsymbol{\theta}) = \sum_{i=1}^n J_i(\boldsymbol{\theta})$$

- Thus

$$\nabla J(\boldsymbol{\theta}) = \sum_{i=1}^n \nabla J_i(\boldsymbol{\theta})$$

---

---

# STOCHASTIC GRADIENT DESCENT (SGD)

- In GD, we calculate the gradient using all of the data
  - For large  $n$ , each step in the parameter space  $\theta$  takes a lot of computations
  - Thus learning can occur slowly
- For SGD, we estimate the gradient using a small random sample of training inputs
  - Speeds up gradient descent and learning

---

# STOCHASTIC GRADIENT DESCENT

- Pick a random set of  $m < n$  training inputs
  - Referred to as a *mini-batch*
- Estimate the gradient using the samples from the mini-batch and take a step in the direction of the negative gradient
- If  $m$  is large enough, then the estimated gradient will be roughly equal to the gradient using the full data

---

# STOCHASTIC GRADIENT DESCENT SUMMARY

A training *epoch*:

1. Pick a random subset of the training data
  - Referred to as a mini-batch
2. Update the parameters using the gradient estimates from the mini-batch
3. Pick another random mini-batch from the remaining training points and repeat step 2
  - Repeat until all training inputs have been used

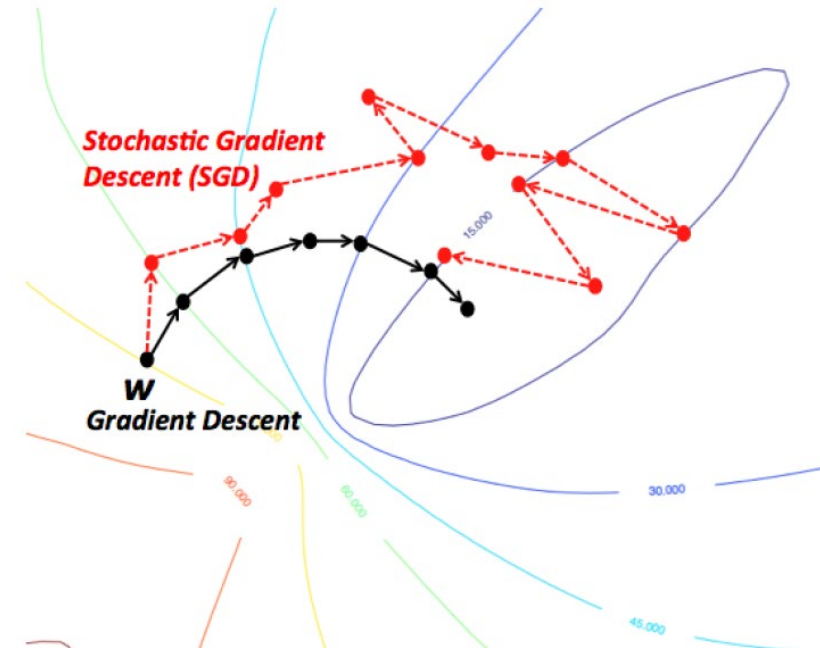
Repeat multiple epochs until stopping conditions are met

---

---

# ANALOGY TO POLITICAL POLLING

- Much easier to carry out a poll than run a full election
- Similarly, it's much easier to estimate gradients from mini-batches than the entire training set
- Downside: gradient estimates will be noisier in SGD
- Generally ok as we only need to move in a general direction that decreases  $J$ 
  - This can be an advantage when the problem is nonconvex
- In practice, SGD (or its variants) are used extensively in learning neural networks



---

# GRADIENT DESCENT VARIATIONS

- Stochastic Gradient Descent (SGD)

- How it works:
  - Uses one random training instance at each step
  - Computes the gradient from that single instance
- **Pros:**
  - Fastest per update (very small computation)
  - Randomness helps escape local minima on complex cost surfaces
- **Cons:**
  - Updates are noisy; the algorithm never fully “settles”
  - True minimum is rarely reached—it oscillates around it

- Mini-Batch Gradient Descent

- How it works:
    - Uses a small random subset (mini-batch) at each step
    - Computes the gradient from that subset
  - **Pros:**
    - More stable than SGD (less noisy updates)
    - Faster than full batch GD on large datasets
    - Leverages vectorized operations for efficiency
  - **Cons:**
    - Still random, so convergence is not perfectly smooth
    - Requires tuning the batch size (too small = noisy, too large = slow)
-

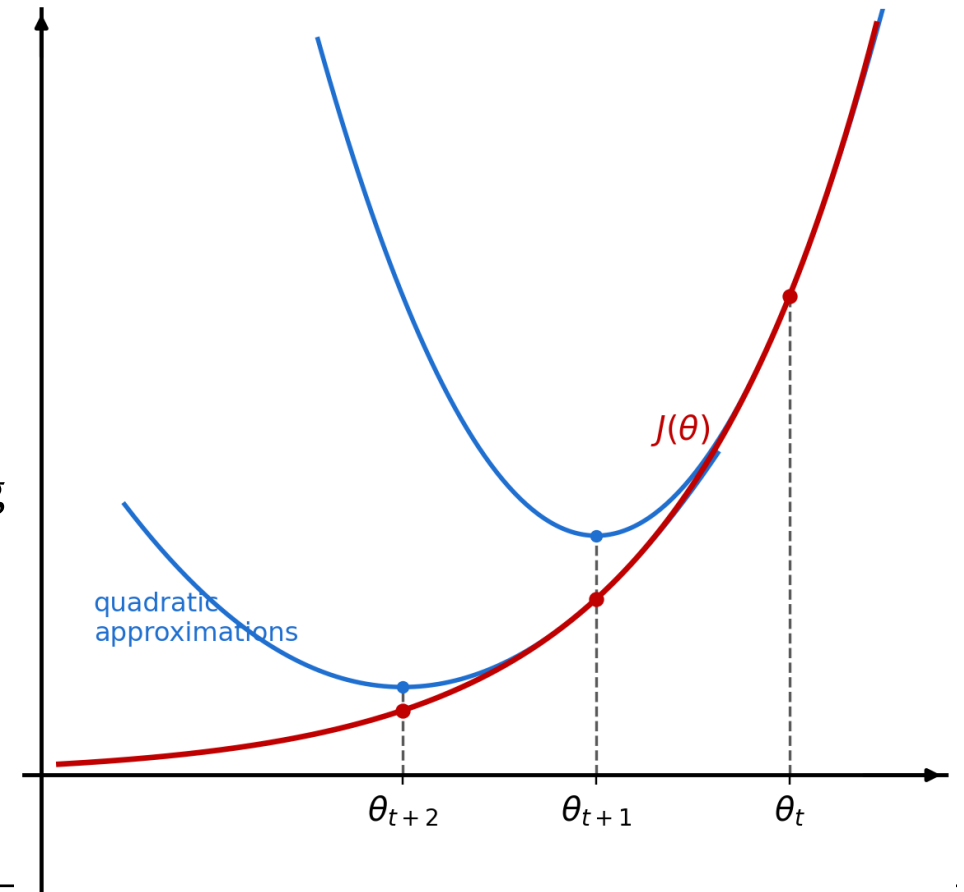
# NEWTON'S METHOD

- Goal is to solve  $\nabla J(\boldsymbol{\theta}) = \mathbf{0}$  for an objective function  $J$
- Newton's method AKA Newton-Raphson:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - (\nabla^2 J(\boldsymbol{\theta}_t))^{-1} \nabla J(\boldsymbol{\theta}_t)$$

- Newton's method can be viewed as minimizing second order approximation:

$$J(\boldsymbol{\theta}) \approx J(\boldsymbol{\theta}_t) + \nabla J(\boldsymbol{\theta}_t)^T (\boldsymbol{\theta} - \boldsymbol{\theta}_t) + \frac{1}{2} (\boldsymbol{\theta} - \boldsymbol{\theta}_t)^T \nabla^2 J(\boldsymbol{\theta}_t) (\boldsymbol{\theta} - \boldsymbol{\theta}_t)$$



---

# NEWTON'S METHOD: INTUITION

- **GD fits a line; Newton fits a parabola**
  - Jumps straight to the minimum of its quadratic model of  $J$
- **The Hessian is an automatic learning rate**
  - $(\nabla^2 J)^{-1}$  shortens steps where curvature is high — no  $\alpha$  to tune
- **Fast when it works**
  - Exact in one step on a quadratic; quadratic convergence near a min
- **But expensive and fragile**
  - $O(d^3)$  per step; Hessian may not be PD far from a min
  - Fixes: damping, trust regions, quasi-Newton (BFGS, L-BFGS)



---

# GROUP EXERCISE

- One advantage of Newton's method is that it converges to a minimum in relatively few steps.  
What is a disadvantage of Newton's method?
  - Answer: You have to invert a matrix (the Hessian) at each step
    - This can be very slow for high-dimensional data
  - Thus in practice, Newton's method is rarely used
  - However, Newton's method has provided inspiration for other optimization techniques that are faster to compute
-

---

# GRADIENT DESCENT VARIATIONS

- Adam Optimizer
- How it works:
  - Maintains two moving averages:
  - First moment (m): average of gradients (like momentum).
  - Second moment (v): average of squared gradients
  - Applies bias correction to both moments.
  - Update rule:

$$\beta_{new} = \beta_{old} - \alpha \frac{\hat{m}}{\sqrt{\hat{v} + \epsilon}}$$